

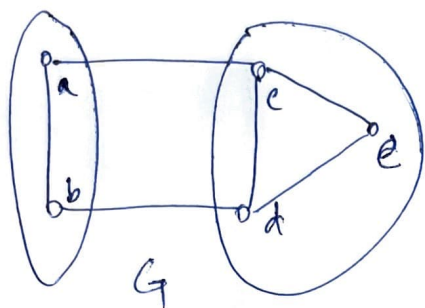
Minimum Spanning tree

(1)

Prims algorithm

* the notion of spanning tree is only applicable to undirected graphs.

So let me define the notion of a cut in an undirected⁺ graph. A cut in a graph $G = (V, E)$ is a partition of the vertex set into two parts.



For example

$\{a, b\} \{c, d, e\}$ is a cut in the graph G .

In a graph there could be many other cuts
eg. $\{a, c, d\} \{b, e\}$

or I could have only one vertex say c on one side and the other 4 vertices on the other side

$\{c\} \{a, b, d, e\}$

How many cuts can there be in a graph?

well: it is 2^{n-1} . How did you get this value?

well every vertex has two choices either it can be in the left part or in the right part. But then

we are repeating each possibility.

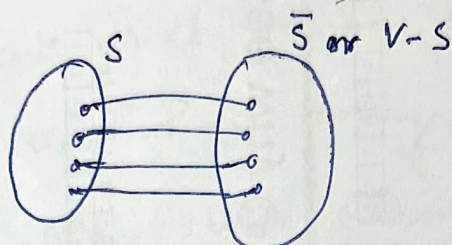
cut edges: edge which has one endpoint in one side and the other endpoint in the other side of partition.

②

we need this notion and how do we use this notion in designing the algorithm

Suppose I take a cut of the given graph.

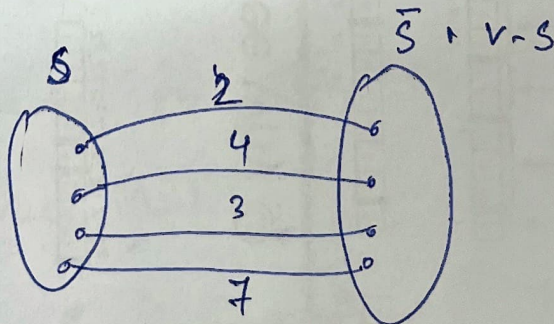
$G(V, E)$



Now let me look at all the edges which are in the cut. Recall we trying to compute MST.

So I am now going to be ~~talking~~ ^{talking} of ~~of~~ a particular property of MST.

Let us take the cut and let us look at the edges in the cut and also let us look at their ~~edges~~ ^{length} also



we are again assuming that all the edge lengths are distinct. This is just to ease all my arguments, to simplify my arguments.

③

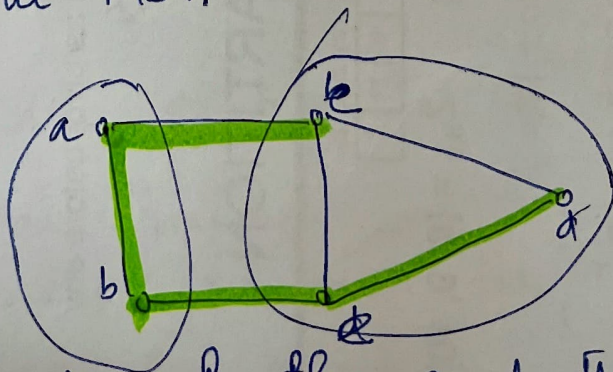
all the arguments algorithm everything work even when edge lengths are not distinct. So just to simplify my presentation here, that we are going to assume that edge lengths are distinct. Recall that when edge lengths are distinct we have only one min. spanning tree. Now the claim is that, the edge in the cut which has smallest weight must be a part of the spanning tree.

Let us write down the claim more formally

Claim: For any cut $(S, \bar{S})$ the min edge in the cut belongs to the MST.

It is not necessary that only the min edge will be part of the MST. ^{More} ~~more~~ than one edge in ~~the~~ on the cut can be part of the MST.

Ex.



I could have edge lengths such that these or edges in the MST, and you see both in

(4)

edges in the cut appears in the MST.

So for any cut it is not necessary that only one edge in the cut will be part of the spanning tree, there could be more than one edge in cut that appears in the spanning tree.

But all we are saying ^{here} is that the min edge in the cut ~~will~~ must belong to the MST. So in the figure the edge with length 2 will have to be in the MST. If it is not there in the spanning tree then it is for ~~sure~~ ^{sure} that, the tree is not a MST.

Why?

Proof:

By contradiction: Let T be a MST which does not contain ~~an~~ edge e (the one with min length)

~~what~~ we can do now is we can add e to the ~~or~~ tree. To ~~of~~ As a result cycle will be formed.

Why? because the end points of e are already connected ~~also~~ via some other

(5)

path. Now the path that goes from u to v must use one of the ~~path~~^{edges} in the cut. ~~(2)~~ So the cycle that has formed as a result of the inclusion of the edge e must include one edge at least in the cut. But all these edge has length ~~more~~ more than that of e . I will remove ~~the~~ the edge from the cut and add ~~that~~ e , that will reduce the cost of the tree.

Formally,

Addition of e to T forms a cycle
Say C .

C contains at least one edge of the cut other than ~~the~~ edge e
 $\Rightarrow C$ contains an edge of length more than the length of e

By removing this edge from $T \cup \{e\}$ ~~we~~
we get a smaller tree;
a contradiction

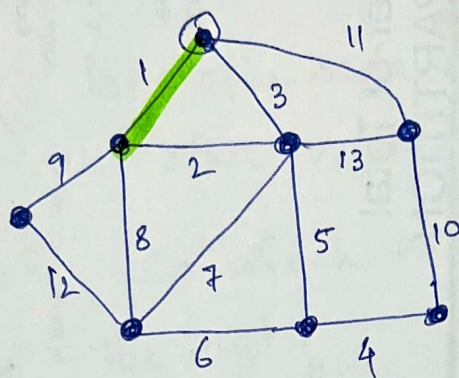
Because we assume that T was a MST.

Now how are we ~~are~~ going to use this claim in Prim's algorithm.

⑥

Prims algorithm ^{exploit} ~~uses~~ this simple fact that if you take any cut the min edge in the cut will be part of the MST. always. Prim's algorithm is ~~essentially~~ built around this simple fact.

Let's understand what ~~pru~~ Prim's algorithm does.



Let say this is my graph on which we are going to illustrate Prim's algorithm

Now lets see how does Prim's algorithm works

First we start from some vertex and we call that the root vertex

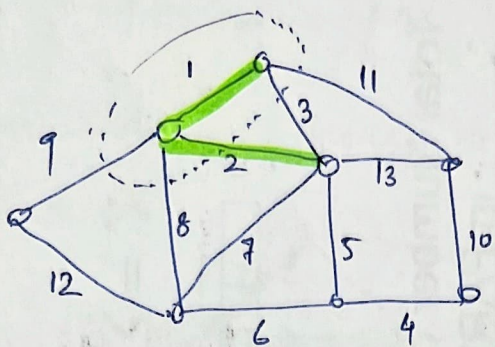
- The first partition/cut that I am considering is the root versus everyone else. root is on one side and everyone else is on the other side.

- So which edge ^{of the cut} has to be part of the MST - edge of length 1.

So I include this in to my MST

(7)

Now what is the cut that I am going to consider next.



The encircled portion going to in one side of the cut and the other side of the cut is going to include all other vertices.

So which are the edges now are part of the cut?

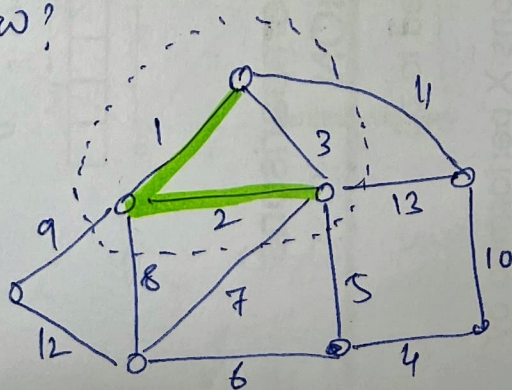
they are $\{9, 8, 2, 3, 11\}$

which one is smallest?

2

So we know for a fact the edge 2 has to be a part of the MST.

Now what is the cut that we are going to consider now?

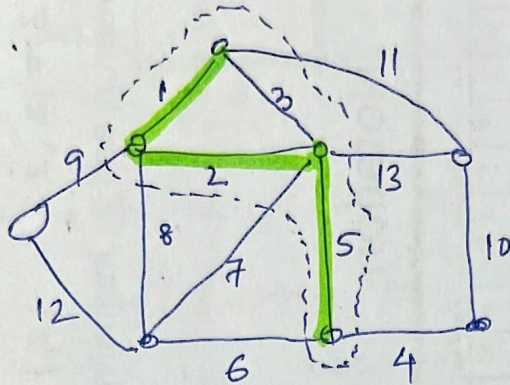


The encircled portion in one side everything else in the other side.

⑧

now which are the edges in the cut that we are going to consider are $\{9, 8, 7, 5, 13, 11\}$

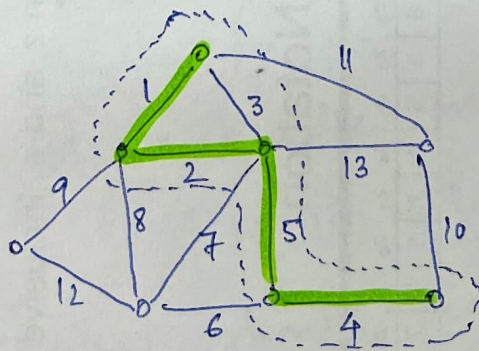
so smallest of there is 5, ~~8~~ and so it gets included.



Now the cut gets extended and become as above.

Now the cut edges that we are going to consider are $\{9, 8, 7, 6, 4, 13, 11\}$

and smallest of them is 4, so it is included in the MST. and the new cut that we get is as follows.



which are the edges in the cut

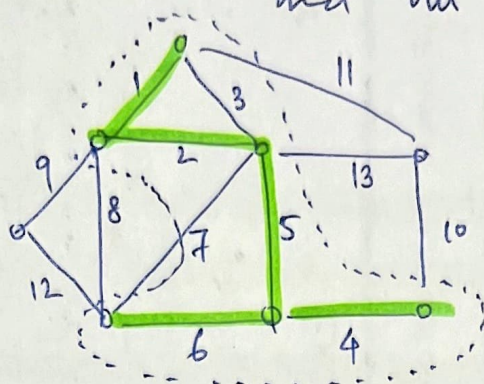
$\{11, 13, 10, 6, 7, 8, 9\}$

Smallest among them is 6, so it's

(9)

... this

included in the MST and the new cut now becomes.



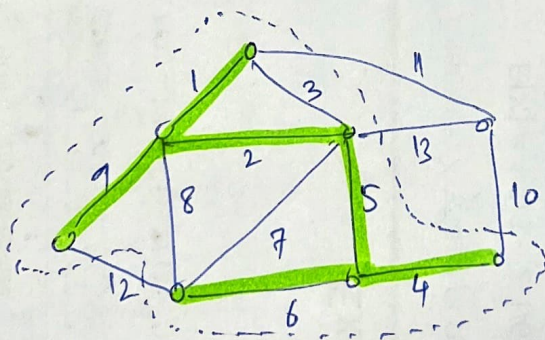
Now which are the cut edges.

is 8 in the cut?
no because both of its end points
are on the same side

So it is $\{9, 12, 10, 13, 11\}$

So ~~we~~ 9 is picked and the cut

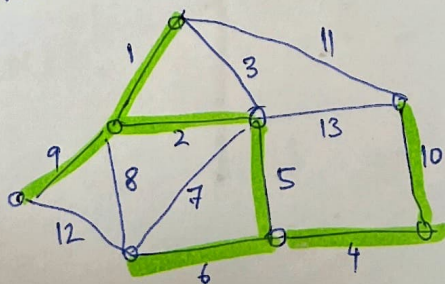
becomes



Now the cut edges are

$\{10, 11, 13\}$

and we should pick 10.



and we
are done
as all the
vertices are
now included.

(10)

use this

We have done the proof effectively, Not effectively we have done the proof. Because what did we say? We proved that this claims that the minimum edge across the cut has to be part of the min spanning tree and that is the edge we are picking at every step. So this is the min spanning tree you will get the same tree since this is unique as we have assumed edge lengths are distinct.

You would get the same tree if you apply the Kruskal algorithm.

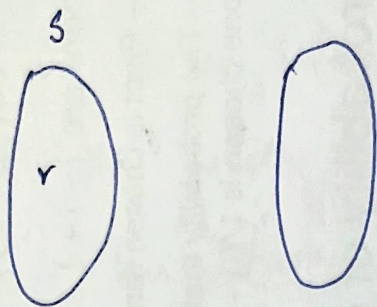
So I am not going to write the pseudo code for this but you understand how the algorithm works. very simple

So this is the idea we have so let us see try and see how you'd implement this algorithm. So what is that you have to maintain of course you have a data structure for the graph. Now what is the operation that you have to do at every step.

At any point you have the

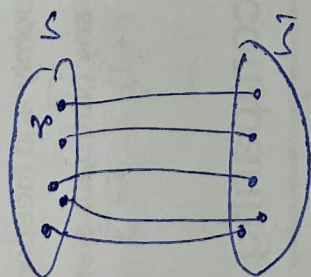
(11)

following



S is one side of the cut and it must contain root. It contains all the vertices that are reached from the root. So the root is always part of the set S , and remaining vertices $v \in V - S$ are in the other part. ^{So we have to maintain collection of values which are 0 or 1.} This can be done very easily by keeping one bit with every vertex. If this bit is 1 then that means it is in S and if it is 0 then it is on the other side i.e. $\bar{S}$.

Now what is that we have to do at every step? I have to look at all the edges

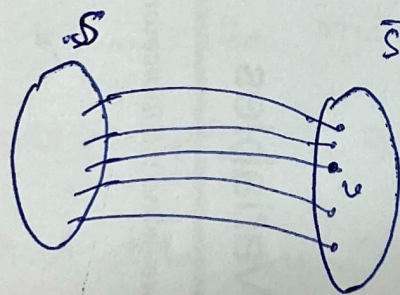


in the cut and find the min. How can I go on to do this?

A naive way would be to look at all the vertices in S look at their edges going

(12)
nt of S . For each edge you see whether the other
end point is in S or not. If it is not in S then
you look at its length and you look at all these
edges and find the min. So ~~how~~ much time
does that take? you are going to each vertex looking
at all its adjacent edges. So what is the
maximum time it could take
 $O(n, m)$

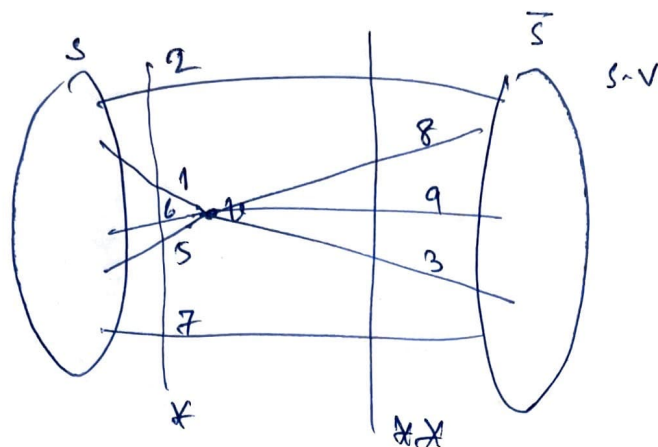
let us look at something clearer. What would be
the cleaner way of doing this?



I have the set S and I also have the complement $\bar{S}$
and there is the vertex v in $\bar{S}$. Why I have
~~single~~ ^{single} this vertex out? because this ~~vertex~~ is the
vertex which is now going to go from $\bar{S}$ to
 S .

(3)

This time you have figured out the min of all the cut edges. Now what is going to happen?



Some edges are going to go from u to S , and some edges are going from u to T , there are of course some edges which go from S to T .

Now we know the min of ~~the~~ edges because u is on the T side, when I move u to S I want to find out the min of ~~the~~ edges, what will I do? ~~compa~~

Suppose I just kept track of min of ~~the~~ edges. Nothing else.

Let's say min is 1, the min has to be an edge incident to u , that you must understand, because that is why we have picked u .

So this is 1 this is 2 this 4, (now the edges)

and then there are 8, 9, 3 let's say. ~~Now the new~~
~~min~~ was the earlier the minimum was 1 and now
the new min is 2. So if I just know the min
it is not going to be useful. So you want me to
keep ~~the~~ track of the second min also?

It is something that we have found out at
this step. So we know the minimum that is coming to
this vertex. So suppose, we keep track of ~~the~~ the min
to each vertex of the vertex is n ~~to~~ 3.

So what data structure should we use to keep
these edges? A heap? Min heap? So, that could
be one possibility. ~~I have a heap~~. So heap is
the data structure ⁱⁿ which ~~with~~ I can put some
elements, each one has 7 then has certain priority
or certain key and it will give me, I can
use an operation called delete min. which will
remove min element. I can find out what
the min element is in the heap in constant
time. I can insert element into the heap.
I can also remove element from the heap.

(15)

I can do all this operation and except for find min all operation take $\log n$ time. Find min takes constant time. Everyone ~~at~~ remembers atleast this much.

So suppose I were to keep a heap here, a heap which contains these edges 2, 1, 6, 5, 7. then using find min I can find out what is the min edge. Once I know the edge, I know the other end point of vertex, I know both the endpoints of the edge, I also know which vertex to bring in where I have to bring in this vertex I ~~look~~ look at all the edges incident at this vertex & the edges which are going in S, I have to remove them from the heap, and the edges which are going from u to $\bar{S}$, I have to add them in the heap. heap is a good data structure for search.

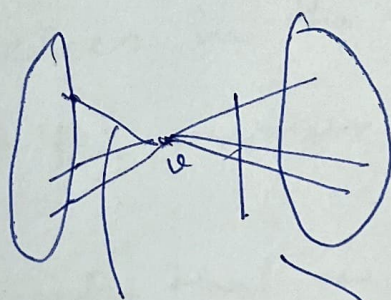
So how do you find out where the edge ~~is~~ is in the heap? once again the same thing.

(16)

when v comes in we insert 3, 9, 8 and 8 and I
am going to delete 1, 6, & 5 and I know when exactly
1, 6 and 5. and this is how I do the implementation.

So at any point what does the heap contain?
all the edges of the cut. exactly those edges
and nothing else. It is best to keep it clean so
that you know what is happening. Now
how many operations are required in the heap

So let us look at the heap operations.



removal has to be added

Each removal and ~~for~~ add takes $\log n$ time

So how much time am I spending

$\text{degree} \times \log n$ for each vertex that I process

d

(17)

Q $d(v) \cdot \log n$

and this has to be summed over all the vertices because I process each vertex exactly once

So
$$\sum_v d(v) \log_2 n$$

I ~~is~~ is the number of element in the heap which could be m

So I should write
$$\sum_v \log m$$

Number of elements in the heap which is the number of edges in the cut, which could be as much as the number of edges. (worst case)

at another operation that we need to do is ~~find~~

Find min Number of vertices time is n time

So - If we also write to add that

$$\sum_v d(v) \log m + n \cdot O(1)$$

so let us write the pseudocode

S is an array

$$S[v] = 1 \text{ if } S[v] \in S \\ 0 \text{ otherwise}$$

What is this S , S is the left side of the cut, not visited but reached

So this one data structure we are going to have
Another data structure that we are going to have
is heap

Initially, I pick a vertex as the root
or $S[v] = 0$

$$S[v] = 1$$

so this is my initialization

No what should I do, I should insert the edges incident at the root into the heap

the incident to v do
 $H.insert(e)$

while ! $H.empty$ do {

$f = H.findmin()$

let v be the endpoint of f if $S[v] = 0$

for all e adjacent to v do
(v, w)